



CYERSHIELD

Project Report

Course Code :- CSE2006

Slot :- A11+A12

Registration Number :- 25BAI10594

Submitted By :- Kunjan Khore

Submitted To :- Prof.MUTHUSUNDARI

Github repository link :- <https://github.com/Kunjan-2006/Cyber-Shield>

INDEX

S. No.	Contents	Page
1	Introduction	3
2	Objectives	3
3	Problem Statement	3
4	Functional Requirements	3
5	Non-Functional Requirements	4
6	System Architecture	4
7	Design Diagrams	5
8	Design Decisions & Rationale	7
9	Implementation	7
10	Screenshots and Results	8
11	Testing	11
12	Challenges Faced	12
13	Learning & Key Takeaways	12
14	Future Enhancements	12
15	Conclusion	12
16	References	13

1. Introduction

CyberShield is a college-level Java application that we developed to understand how multiple cybersecurity incidents can be managed when many incidents are reported at the same time and there are only a limited number of analysts and resources available. The application follows a simple process where incidents are reported, checked, prioritized, assigned to analysts, and finally resolved.

For the interface, we used JavaFX and FXML. The project follows an MVC/layered structure, uses SQLite to store the data, and uses Java PriorityQueue to prioritize incidents according to their severity. CyberShield is only an academic project and is not connected to any real cybersecurity system.

2. Objectives

- Report and validate cybersecurity incidents.
- Assign Critical, High, Medium or Low severity.
- Prioritize incidents effectively.
- Match incidents with available analysts based on required skills.
- Track investigation and response operations.
- Record incident evidence.
- View unified live dashboard statistics.

3. Problem Statement

Cybersecurity operations can receive many incidents at the same time while analyst capacity is limited. CyberShield addresses this academic problem by prioritizing incidents according to severity and checking analyst skill availability before assigning response work.

4. Functional Requirements

ID	Requirement	Description
FR1	Incident Reporting	Create and validate incidents.
FR2	Severity Assignment	Critical, High, Medium and Low.
FR3	Priority Processing	PriorityQueue-based ordering.
FR4	Analyst Management	Add analysts and track availability.
FR5	Skill Matching	Match incidents to required analyst skills.
FR6	Affected Systems	Track compromised systems and recovery.
FR7	Evidence Management	Record logs, screenshots and artifacts.
FR8	Response Operations	Track resolution lifecycle.
FR9	Dashboard	Display incident and analyst statistics.

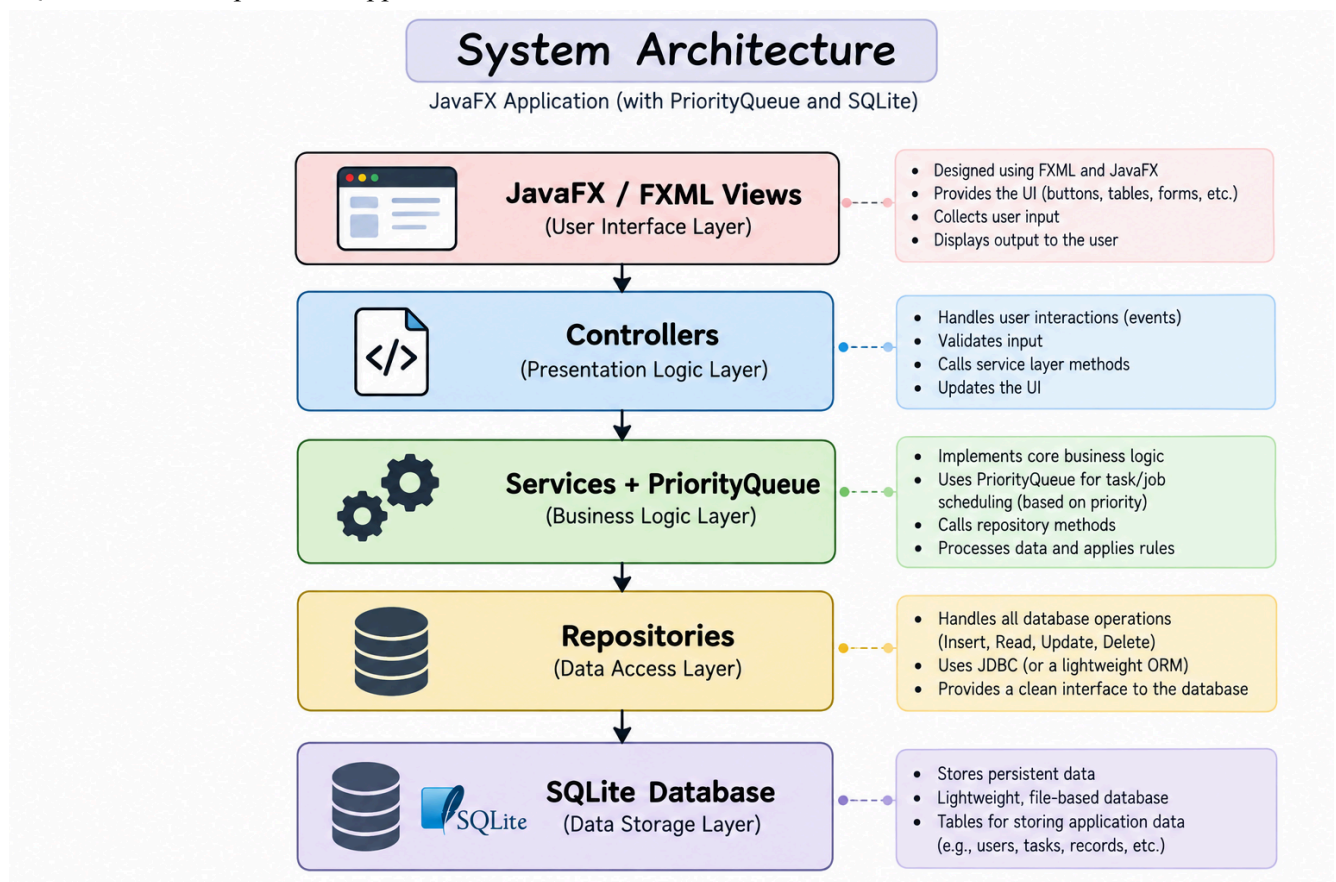
5. Non-Functional Requirements

- Usability – clear JavaFX navigation.
- Maintainability – separated UI, service and repository layers.
- Reliability – persistent SQLite storage and structured data access.
- Performance – PriorityQueue gives efficient access to the next priority incident.
- Data Integrity – consistent incident, analyst, system and evidence records.
- Testability – JUnit 5 for core logic.
- Portability – Java 17 and Maven based project.

6. System Architecture

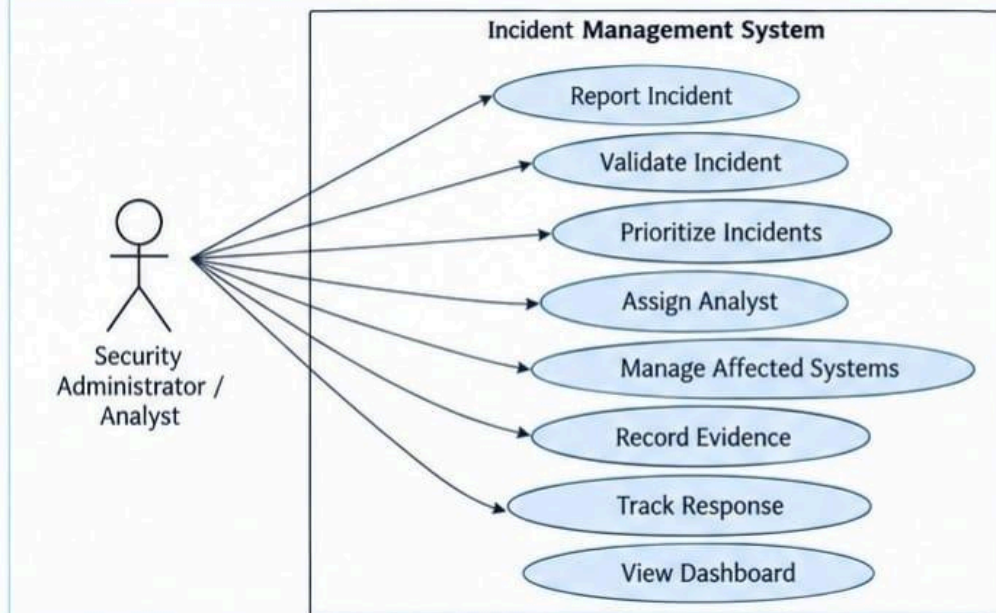
CyberShield follows a clean MVC/layered architecture:

- JavaFX/FXML Views – graphical interface.
- Controllers – handle UI interaction.
- Services – business rules, prioritization and analyst matching.
- Repository – SQLite connection and queries.
- SQLite Database – persistent application data.

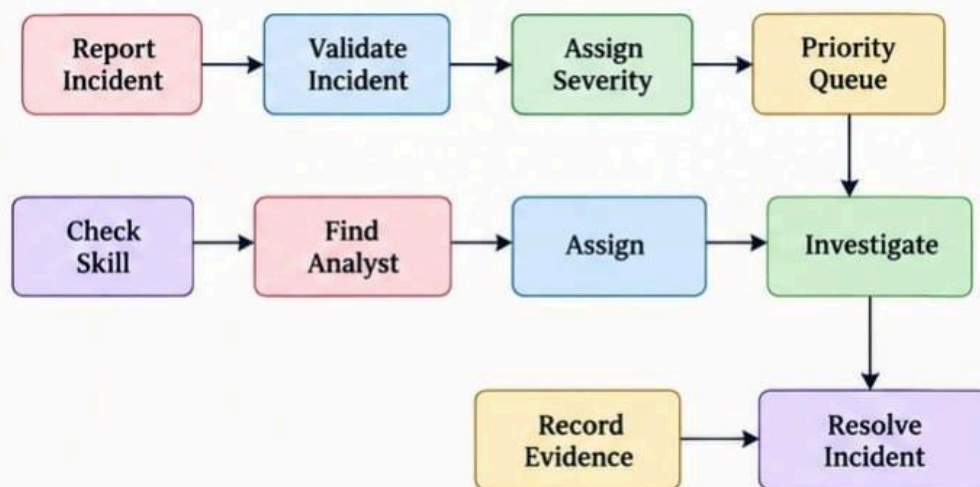


7. Design Diagrams

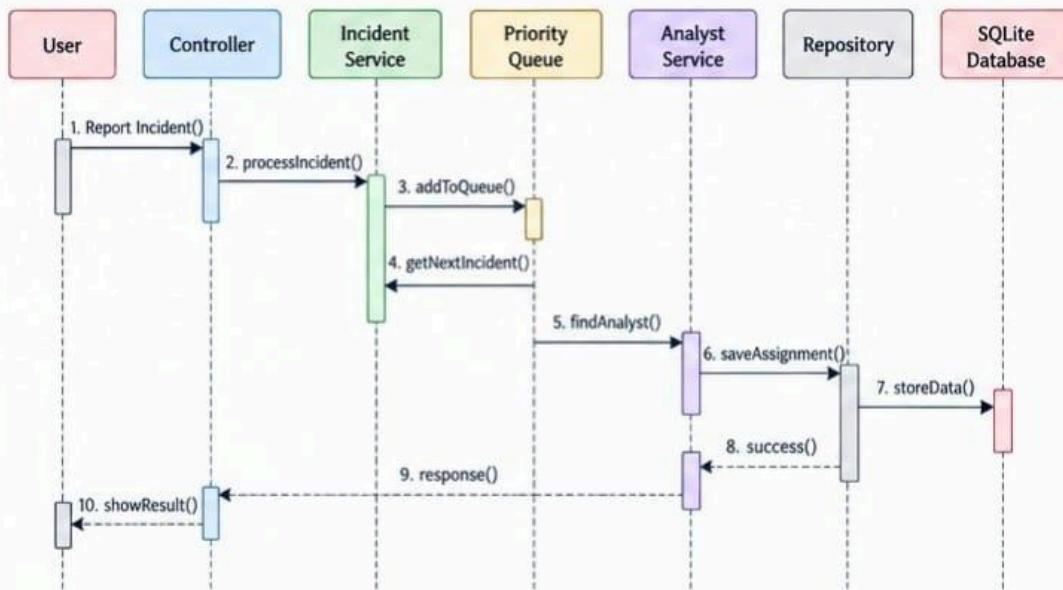
7.1 Use Case Diagram



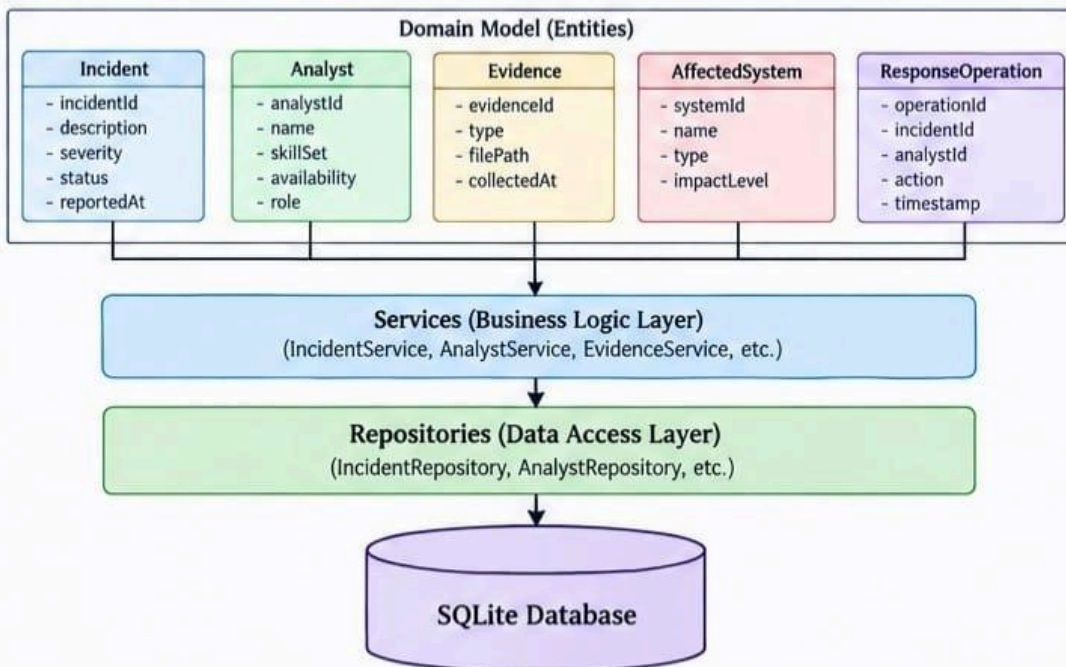
7.2 Workflow Diagram



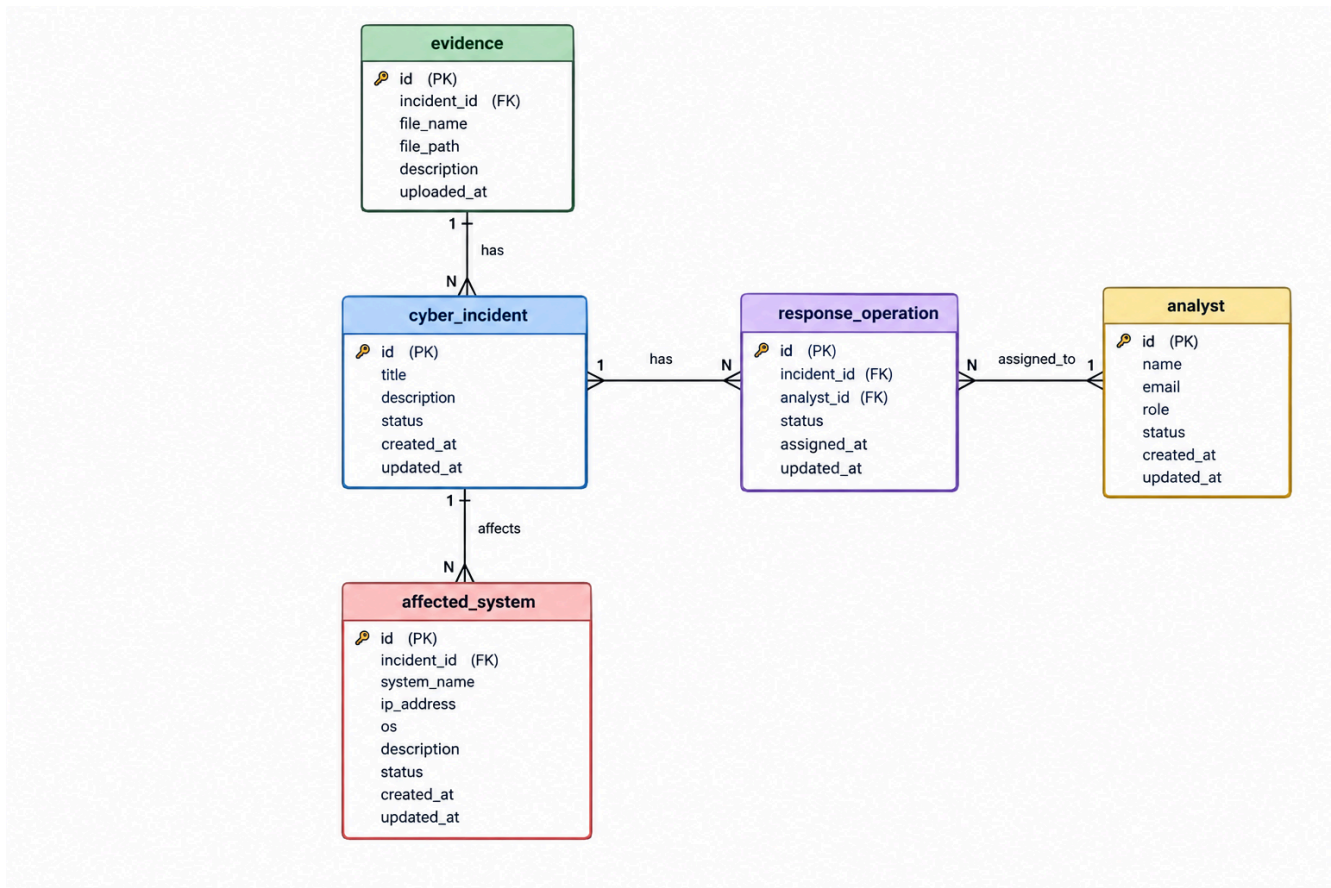
7.3 Sequence Diagram



7.4 Class / Component Diagram



7.5 ER Diagram



8. Design Decisions & Rationale

- PriorityQueue – used to keep the pending incidents arranged according to their priority automatically.
- Severity ordering – incidents are arranged as CRITICAL > HIGH > MEDIUM > LOW.
- Reported-time tie-breaker – if two incidents have the same severity, the one reported earlier is considered first.
- Skill matching – checks whether the analyst has the required skill before assigning an incident.
- MVC/layered architecture – keeps the interface, business logic, and database-related work separate.
- SQLite – chosen because it is a lightweight relational database and works well for a desktop academic application.
- JUnit 5 – used to automatically test and verify the main rules of the system.

9. Implementation

9.1 Priority Processing

Pending incidents are stored in a Java PriorityQueue. The comparator orders Critical, High, Medium and Low incidents, with reported time used as a tie-breaker.

9.2 Incident Assignment

processNextIncident() peeks at the highest-priority incident and attempts to find an available analyst with the required skill before removing the incident from the queue.

9.3 Persistence

SQLite is used for persistent storage and repository classes that handle database operations separately from controllers and business services.

9.4 Technologies

- Java 17
- JavaFX / FXML
- Maven
- SQLite
- JUnit 5
- PriorityQueue

10. Screenshots and Results

Figure 10.1 – Main Dashboard

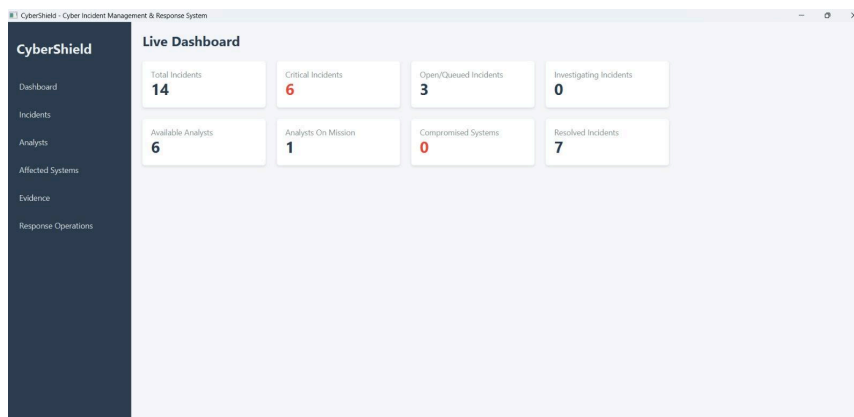


Figure 10.2 – Incident Management / Create Incident

Incident Management

Reporter: Incident Type: Severity:

Affected System ID: Required Skill:

Description:

Success

Incident created and queued successfully.

ID	Reporter	Type
----	----------	------

Figure 10.3 – PriorityQueue / Incident Prioritization

ID	Reporter	Type	Severity	Required Skill	Status
INC-TEST-16-9367...	Rep	MALWARE	HIGH	Skill	CLOSED
INC-TEST-17B-936...	Rep	MALWARE	HIGH	Skill	CLOSED
INC-TEST-17A-936...	Rep	MALWARE	HIGH	Skill	CLOSED
TEST-93676670316...	John Doe	MALWARE	CRITICAL	Malware Analysis	ASSIGNED
INC-D6B5E76F	Toji	DATA_BREACH	CRITICAL	Digital Forensics	QUEUED
INC-TEST-16-1151...	Rep	MALWARE	HIGH	Skill	CLOSED
INC-TEST-17B-115...	Rep	MALWARE	HIGH	Skill	CLOSED
INC-TEST-17A-115...	Rep	MALWARE	HIGH	Skill	CLOSED
TEST-11513168712...	John Doe	MALWARE	CRITICAL	Malware Analysis	ASSIGNED
INC-A2E75C18	Toji	MALWARE	CRITICAL	Malware Analysis	CLOSED
TEST-98407391727...	John Doe	MALWARE	CRITICAL	Malware Analysis	ASSIGNED
INC-TEST	Rep	MALWARE	HIGH	Skill	QUEUED
TEST-93013196045...	John Doe	MALWARE	CRITICAL	Malware Analysis	ASSIGNED
INC-123	Rep	MALWARE	HIGH	Skill	QUEUED

Figure 10.4 – Analyst Management and Availability

Analyst Management

Analyst ID: Name:

Skill: Experience Level:

Status:

ID	Name	Skill	Experience	Status
T-ANALYST-1	Test Analyst	Malware Analysis	Mid	UNAVAILABLE
T-ANALYST-2	Net Analyst	Network Security	Mid	AVAILABLE
A-TEST	A	Skill	Mid	AVAILABLE
A-001	Malware Specialist	Malware Analysis	Senior	ON_MISSION
A-TEST-17-115...	A17	Skill	Mid	AVAILABLE
A-TEST-16-115...	A16	Skill	Mid	AVAILABLE
A-TEST-17-936...	A17	Skill	Mid	AVAILABLE
A-TEST-16-936...	A16	Skill	Mid	AVAILABLE

Figure 10.7 – Response / Resolution Status

Response Operations

Select an operation from the table, then apply a valid status transition below.

New Status:

Resolution Notes:

Update Status

Refresh Table

Operation ID	Incident ID	Analyst ID	Start Time	End Time	Status	Notes
OP-51F728C4	TEST-93013196...	T-ANALYST-1	2026-09-17T15:10:30.3...		ASSIGNED	
OP-TEST-93015...	INC-TEST	A-TEST	2026-09-17T15:10:30.5...		INVESTIGATING	Started investigation
OP-70FF0374	TEST-98407391...	T-ANALYST-1	2026-09-17T15:19:29.7...		ASSIGNED	
OP-TEST-98409...	INC-TEST	A-TEST	2026-09-17T15:19:29.9...		INVESTIGATING	Started investigation
OP-71A0D7C4	INC-A2E75C18	A-001	2026-09-17T15:29:37.3...	2026-09-17T15:37:29.8...	CLOSED	Simulated malware incident fully cl...
OP-66AF353B	TEST-11513168...	T-ANALYST-1	2026-09-17T15:47:22.1...		ASSIGNED	
OP-TEST-17A-1...	INC-TEST-17A-...	A-TEST-17-115...	2026-09-17T15:47:22.4...	2026-09-17T15:47:22.4...	CLOSED	
OP-TEST-17B-1...	INC-TEST-17B-...	A-TEST-17-115...	2026-09-17T15:47:22.4...	2026-09-17T15:47:22.5...	CLOSED	
OP-TEST-11513...	INC-TEST	A-TEST	2026-09-17T15:47:22.6...		INVESTIGATING	Started investigation
OP-TEST-16-11...	INC-TEST-16-1...	A-TEST-16-115...	2026-09-17T15:47:22.7...	2026-09-17T15:47:22.8...	CLOSED	
OP-802D8B1E	TEST-93676670...	T-ANALYST-1	2026-09-18T14:36:43.3...		ASSIGNED	
OP-TEST-17A-9...	INC-TEST-17A-...	A-TEST-17-936...	2026-09-18T14:36:43.5...	2026-09-18T14:36:43.5...	CLOSED	
OP-TEST-17B-9...	INC-TEST-17B-...	A-TEST-17-936...	2026-09-18T14:36:43.5...	2026-09-18T14:36:43.6...	CLOSED	
OP-TEST-93676...	INC-TEST	A-TEST	2026-09-18T14:36:43.6...		INVESTIGATING	Started investigation
OP-TEST-16-93...	INC-TEST-16-9...	A-TEST-16-936...	2026-09-18T14:36:43.7...	2026-09-18T14:36:43.7...	CLOSED	

11. Testing

Core testing should verify prioritization, tie-breaking, analyst matching, availability, validation and database operations.

Test Area	Expected Result	Actual Result
PriorityQueue	CRITICAL before HIGH before MEDIUM before LOW	PASS – incidents are processed according to priority order.
Tie-breaker	Earlier reported incident first	PASS – earlier reported incident is selected first when priority is the same.
Skill matching	Required-skilled analyst selected	PASS – analyst with the required skill is assigned.
Availability	On-Mission analyst not assigned	PASS – analysts already on active missions are not assigned to new operations.
Validation	Invalid data rejected	PASS – invalid input/state conditions are rejected by validation rules.
Database	Records stored/retrieved correctly	PASS – incident, operation, and analyst records are stored and retrieved correctly.

12. Challenges Faced

- It was a bit challenging to decide the priority of incidents using severity and time.
- Handling incidents having the same severity was another challenge.
- Finding the right analyst according to the required skills and their availability was difficult.
- We had to make sure that the UI, service, and database parts stayed separate.
- Keeping the incident and response data properly consistent was also important.

13. Learning & Key Takeaways

- Got practical experience with Java PriorityQueue and comparators.
- Understood how MVC and layered architecture are used in an application.
- Learned how SQLite can be used for data storage through repositories.
- Learned how cybersecurity-related entities can be represented using OOP.
- Understood how limited analyst availability can affect the response process.
- Got experience in testing business rules using JUnit 5.

14. Future Enhancements

- Connect the system with a real SIEM.
- Add automatic alert ingestion.
- Add role-based access control.
- Add threat intelligence integration.
- Improve analyst workload balancing.
- Add notifications and incident-response reports.

15. Conclusion

CyberShield is an academic simulation of a cybersecurity incident-management system. It focuses on prioritizing incidents and assigning them to analysts when there is limited analyst availability.

The project uses JavaFX, PriorityQueue, SQLite, layered architecture, and JUnit 5 to bring different software-engineering and data-structure concepts together in a cybersecurity-based project.

The system is made for educational purposes only. It does not connect to real cybersecurity infrastructure or perform actual network scanning.

16. References

- Java 17 Documentation
- JavaFX Documentation
- SQLite Documentation
- JUnit 5 Documentation
- Maven Documentation

Repository: <https://github.com/Kunjan-2006/Cyber-Shield>